

Sun'iy intellekt darajasini baholash (Tyuring testi)

1. Me'yorning paydo bo'lishi: falsafiy jumboqdan pragmatik muammoga

1950 yilda britaniyalik matematik va kriptograf Alan Tyuring *Mind* jurnalida sun'iy intellekt falsafasi bo'yicha eng ta'sirli ishlaridan birini e'lon qildi. Unga "Computing Machinery and Intelligence" ("Hisoblash mashinalari va intellekt") deb nom berildi. Ushbu asar asrlar davomida mutafakkirlar ongini band qilgan fundamental savolga javob berish uchun yozilgan edi: "Mashinalar fikrlashi mumkinmi?".

Tyuring, biroq, tezda xulosaga keldi: bu savolning o'zi aniq ilmiy tadqiqot mavzusi bo'lishi uchun haddan tashqari noaniq va ma'nosiz. "Mashina" va "fikrlash" tushunchalarini aniqlashdagi qiyinchiliklar bu savolga bir xil javob berishni deyarli imkonsiz qiladi. Falsafiy botqoqqa chuqur kirib borish o'rniga Tyuring bu yechilmas dilemmani yanada aniqroq va pragmatik vazifa bilan almashtirishni taklif qildi — u "imitatsiya o'yini" (imitation game) deb atagan o'yin. Bu yondashuv muhokamani sof falsafa sohasidan empirik o'lchash va erishilgan yutuqlarni baholash mumkin bo'lgan sohaga olib o'tdi.

2. Imitatsiya o'yini nima?

Asl versiyada, Tyuringning imitatsiya o'yini uchta ishtirokchini o'z ichiga oladi: erkak, ayol va so'roqchi (odatda erkak). So'roqchining vazifasi — yozma savol-javoblar orqali kim erkak, kim ayol ekanini aniqlash. Erkakning roli ayolni taqlid qilish, ayolning roli esa o'zini himoya qilishdan iborat.

Tyuring keyinchalik o'yinning muhim variantini taklif qildi: agar inson ishtirokchilardan birining o'rniga mashina qo'yilsa-chi? Agar so'roqchi muloqot asosida mashinani insonga o'xshash ishonch bilan aniqlay olmasa, unda biz "mashina fikrlay oladi" degan xulosaga kelishimiz kerak bo'ladi.

Bu o'yin, shunday qilib, mashinaning "fikrlash qobiliyati"ni baholashning dastlabki shakli sifatida xizmat qildi. Savolni noaniq falsafiy so'zlardan aniqroq: inson xulq-atvorini taqlid qilish orqali imtihondan o'tish mumkinmi, degan masalaga ko'chirdi.

3. Test mezonlari: so'zlashuv qobiliyatining markaziy o'rni

Tyuring testining eng muhim jihati shundaki, u intellektni jismoniy harakatlar, sezgi yoki ichki ong emas, balki **til** orqali baholaydi. Test faqat yozma muloqotga asoslanadi, bu esa insonlar va mashinalar oʻrtasidagi barcha tashqi farqlarni yoʻq qiladi.

Bu yondashuvning falsafiy oqibatlari juda katta: agar mashina soʻzlashuv orqali ishonarli darajada odamni taqlid qila olsa, unda unga “intellektga ega” deb baho berish mumkin. Demak, ongning tashqi koʻrinishi (yaʼni, nutqdagi javoblar) uning ichki mohiyatidan koʻra muhimroq deb hisoblanadi.

Tyuring oʻz maqolasida 2000-yilga borib, kompyuterlar oʻrtacha soʻroq qiluvchilarning 30 foizini muvaffaqiyatli aldashi mumkin boʻladi, deb taxmin qilgan edi. Bu oʻsha davr uchun juda jasur bashorat edi.

4. Tanqidlar va cheklovlar

Tyuring testi taʼsirchan boʻlsa-da, u tanqidchilardan ham qochib qutula olmadi. Asosiy eʼtirozlar quyidagilardan iborat:

1. **Chuqurlik oʻrniga yuzaki taqlid.** Mashina testdan oʻtishi mumkin, lekin bu uning haqiqiy tushunishiga kafolat bermaydi. U shunchaki naqshlarni va til oʻyinlarini takrorlashi mumkin. Bu nuqta keyinchalik Jon Searlning mashhur *Xitoy xonasi* tajribasida yaqqol ifoda etilgan.
2. **Cheklangan doira.** Test faqat tilga eʼtibor qaratadi va boshqa muhim jihatlarni – sezgi, motorika, hissiyotlar yoki ongni eʼtiborga olmaydi.
3. **Madaniy va kognitiv tarafkashlik.** Insonni taqlid qilish mezonlari madaniy jihatdan oʻzgaruvchan boʻlishi mumkin. Bir kontekstdagi tabiiy muloqot boshqa kontekstda ishonchsiz koʻrinishi mumkin.
4. **Haqiqiy intellektni emas, balki odamga oʻxshashlikni baholash.** Mashina insoniy javoblarni ishlab chiqishda mohir boʻlishi mumkin, biroq bu uning ongli yoki mustaqil fikrlash qobiliyatiga ega ekanini anglatmaydi.

5. Zamonaviy davrdagi Tyuring testi

Soʻnggi oʻn yilliklarda mashinali oʻqitish, chuqur oʻqitish va katta til modellari rivojlanishi Tyuring testini yanada dolzarb qildi. GPT, LaMDA yoki ChatGPT kabi tizimlar koʻplab kontekstlarda odamga oʻxshash darajada muloqot qila oladi. Bu esa shunday savollarni keltirib chiqarmoqda:

- **Tyuring testi endi juda “oson” bo‘lib qolmadimi?** Agar mashinalar oddiy muloqotlarda odamlarni aldash darajasiga yetgan bo‘lsa, unda bu mezon yetarli emasmi?
- **Yangi testlar kerakmi?** Ba’zi tadqiqotchilar hozirgi paytda murakkabroq baholash usullari talab qilinadi, deb hisoblashmoqda – masalan, tizimning ijodiy muammolarni hal qilish, bilimni umumlashtirish yoki axloqiy qarorlar qabul qilish qobiliyatini tekshirish.
- **Ijtimoiy oqibatlar.** Agar mashinalar odamlarni muloqotda aldashi mumkin bo‘lsa, bu axloqiy va siyosiy savollarni keltirib chiqaradi – masalan, noto‘g‘ri ma’lumot tarqatish yoki manipulyatsiya qilish ehtimoli.

6. Generativ sun’iy intellekt davrida dolzarblik

Generativ modellar, jumladan ChatGPT, DALL·E yoki boshqa multimodal tizimlar, Tyuring testini yangi bosqichga olib chiqdi. Endi gap shunchaki matnli dialog emas, balki ko‘p tarmoqli ijodkorlik — matn, tasvir, audio va hatto video yaratish haqida ketmoqda. Bu esa savolni kengaytiradi:

- Agar mashina nafaqat odamdek suhbatlashsa, balki odamlar yaratgan san’at asarlari va ilmiy matnlarga teng mahsulotlar yaratsa, u holda “intellekt”ning mezoni qay darajada o‘zgaradi?
- Tyuring testi bu murakkab qobiliyatlarni o‘lchash uchun yetarli yoki yangi, yanada chuqurroq testlar kerakmi?

Ko‘plab olimlar bugungi kunda Tyuring testini “boshlang‘ich nuqta” deb hisoblashmoqda, lekin endilikda u yagona yoki eng yaxshi mezon emas. Uning ahamiyati tarixiy va ramziy ahamiyatida qolmoqda, ammo amaliy jihatdan biz mashinalarning kognitiv imkoniyatlarini baholash uchun yanada ko‘p o‘lchovli yondashuvlarga muhtojmiz.

Tyuring testi sun’iy intellekt haqidagi bahslarning markazida 70 yildan ortiq vaqt davomida turibdi. Uning kuchi shundaki, u “mashinalar fikrlashi mumkinmi?” degan mavhum savolni empirik mezon bilan almashtirdi: “Mashina insoniy muloqotni yetarlicha darajada taqlid qila oladimi?”.

Bugungi kunda, generativ sun’iy intellekt davrida, testning cheklovlari yanada ravshanlashdi. U intellektning chuqurligini emas, balki insoniylikning yuzaki ko‘rinishini o‘lchaydi. Shunga

qaramay, uning ahamiyati tarixiy nuqtai nazardan ham, intellekt tushunchasini shakllantirishdagi roli jihatidan ham katta.

Zamonaviy muhokamalar Tyuring testidan voz kechishni emas, balki uni kengaytirishni taklif qilmoqda. Masalan, mashinalarning ijodkorlik, o'z-o'zini anglash, axloqiy fikrlash va ko'p modal qobiliyatlarini baholash.

Shunday qilib, Tyuring testining dolzarbligi davom etmoqda: u bizni mashinalar qachon va qanday qilib insoniy qobiliyatlarga yaqinlashishini aniqlashga undaydi, biroq endi u kognitiv baholashning murakkabroq mezonlari bilan to'ldirilishi kerak.

3 amaliy misol: "Muammoni yechimini topish" — Python kodlari va natijalari

Misol 1: Kenglik bo'yicha qidiruv (BFS) — unweighted graph, Toshkent -> Buxoro misoli

Misol 2: Dijkstra — og'irlikli graf, eng qisqa masofa

Misol 3: A* (grid va manhattan) — to'siqlar bilan qatordan eng qisqa yo'l

Kodni ishga tushiramiz va natijalarini ko'rsatamiz.

```
from collections import deque, defaultdict
```

```
import heapq
```

```
import math
```

```
print("=== Misol 1: BFS (kenglik bo'yicha qidiruv) ===")
```

```
graph = {
```

```
    "Toshkent": ["Samarqand", "Jizzax", "Navoiy"],
```

```
    "Samarqand": ["Toshkent", "Buxoro"],
```

```
    "Jizzax": ["Toshkent", "Buxoro"],
```

```
    "Navoiy": ["Toshkent", "Buxoro"],
```

```
    "Buxoro": ["Samarqand", "Jizzax", "Navoiy"]
```

```
}
```

```
def bfs_shortest_path(graph, start, goal):
```

```
    # Queue stores (node, path_to_node)
```

```
    queue = deque([(start, [start])])
```

```
    visited = set([start])
```

```
    while queue:
```

```
        node, path = queue.popleft()
```

```
        if node == goal:
```

```
            return path
```

```
        for nei in graph.get(node, []):
```

```
            if nei not in visited:
```

```
                visited.add(nei)
```

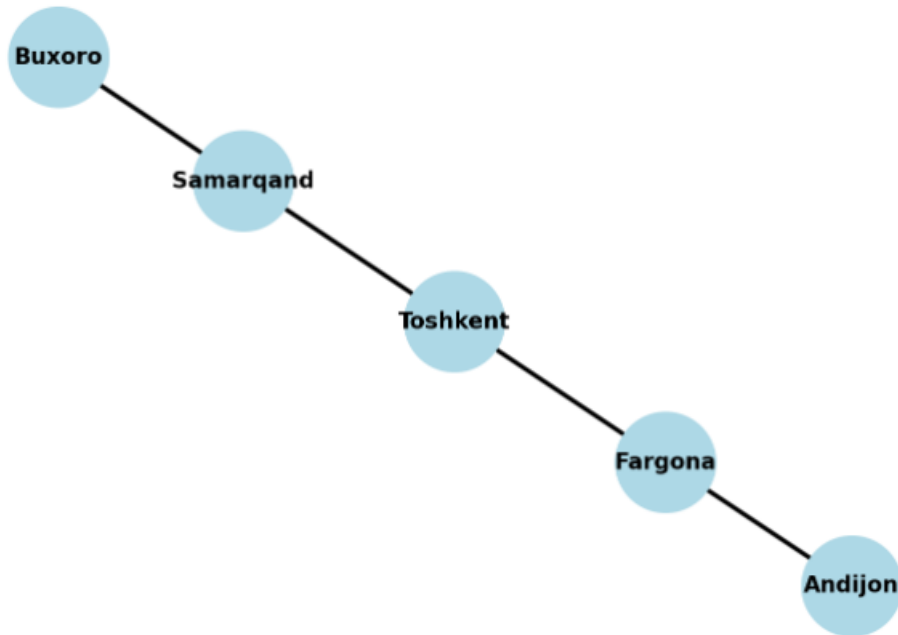
```
                queue.append((nei, path + [nei]))
```

```
    return None
```

```
path = bfs_shortest_path(graph, "Toshkent", "Buxoro")
```

```
print("Toshkent -> Buxoro uchun BFS topgan yo'l:", path)
```

Misol 1: BFS grafigi (og'irliksiz)



```
print("\n=== Misol 2: Dijkstra (og'irlikli graf) ===")
```

```
# Har bir yo'l uchun masofa (km) bilan og'irlik
```

```
wgraph = {
```

```
    "Toshkent": {"Samarqand": 300, "Jizzax": 220, "Navoiy": 360},
```

```
    "Samarqand": {"Toshkent": 300, "Buxoro": 270},
```

```
    "Jizzax": {"Toshkent": 220, "Buxoro": 480},
```

```
    "Navoiy": {"Toshkent": 360, "Buxoro": 600},
```

```
    "Buxoro": {"Samarqand": 270, "Jizzax": 480, "Navoiy": 600}
```

```
}
```

```
def dijkstra(graph, start, goal):
```

```
    pq = [(0, start, [start])] # (dist, node, path)
```

```
    seen = {}
```

```
    while pq:
```

```
        dist, node, path = heapq.heappop(pq)
```

```
        if node == goal:
```

```

    return dist, path

    if node in seen and seen[node] <= dist:

        continue

    seen[node] = dist

    for nei, w in graph.get(node, {}).items():

        nd = dist + w

        heapq.heappush(pq, (nd, nei, path + [nei]))

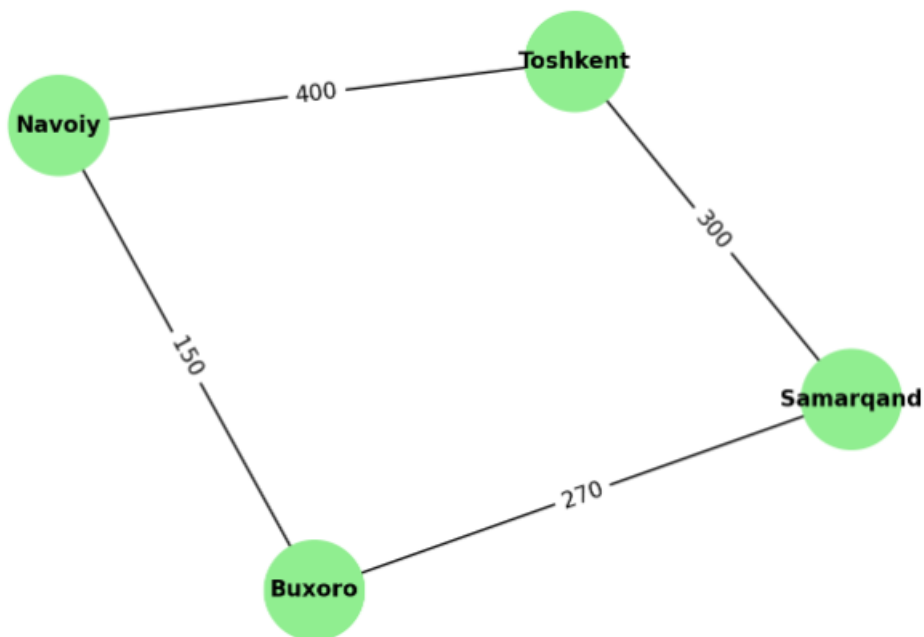
    return math.inf, None

```

```
dist, path = dijkstra(wgraph, "Toshkent", "Buxoro")
```

```
print(f"Toshkent -> Buxoro uchun eng qisqa masofa (Dijkstra): {dist} km, yo'l: {path}")
```

Misol 2: Dijkstra grafigi (og'irlikli)



```
print("\n=== Misol 3: A* (grid, Manhattan heuristic) ===")
```

```
# 6x6 grid misoli. 0 - o'tish mumkin, 1 - to'siq
```

```
grid = [
```

```
    [0,0,0,0,0,0],
```

```

[0,1,1,0,1,0],

[0,0,0,0,1,0],

[0,1,0,0,0,0],

[0,1,0,1,1,0],

[0,0,0,0,0,0]

]

start = (0,0)

goal = (5,5)


def neighbors(pos):

    x,y = pos

    for dx,dy in [(1,0),(-1,0),(0,1),(0,-1)]:

        nx, ny = x+dx, y+dy

        if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]) and grid[nx][ny]==0:

            yield (nx, ny)


def manhattan(a,b):

    return abs(a[0]-b[0]) + abs(a[1]-b[1])


def astar(start, goal):

    open_heap = [(manhattan(start,goal), 0, start, [start])] # (f, g, node, path)

    gscore = {start: 0}

    while open_heap:

        f, g, node, path = heapq.heappop(open_heap)

        if node == goal:

```



```

    return g, path

    if g != gscore.get(node, None):

        continue

    for nei in neighbors(node):

        ng = g + 1

        if ng < gscore.get(nei, math.inf):

            gscore[nei] = ng

            nf = ng + manhattan(nei, goal)

            heapq.heappush(open_heap, (nf, ng, nei, path + [nei]))

    return math.inf, None

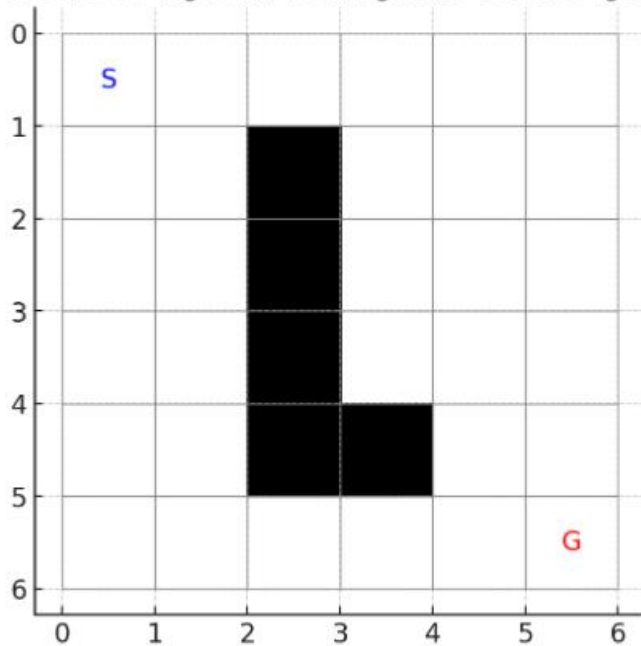
```

```
cost, path = astar(start, goal)
```

```
print("Grid uchun A* natija: xarajat (qadamlar) =", cost)
```

```
print("Yo'l (koordinatalar bo'yicha):", path)
```

Misol 3: A* algoritmi uchun grid (S=start, G=goal)



Qo'shimcha: talabalar uchun izohlar

```
notes = ""
```

Izohlar (o'zbekcha):

- Misol 1 (BFS): og'irliksiz graf uchun eng qisqa yo'l (tugun soni bo'yicha).
- Misol 2 (Dijkstra): har bir qadamning og'irligi (masofa, vaqt yoki boshqa xarajat) bo'lsa, eng kam umumiy og'irlikli yo'lni topadi.
- Misol 3 (A*): agar sizda izchil heuristika (masalan Manhattan) bo'lsa, A* tezroq ishlaydi va maqsadga samarali yetib boradi.

```
""
```

```
print(notes)
```